Search Consistency

saved is not the same as searchable

Watch the 4-min video

In one line: search is **near real time** and, behind a pipeline, **eventually consistent** with the database; force visibility with `refresh=wait_for` only where you can afford it, hide the gap in the product with **read-your-writes**, never trust the index for permissions or stock, and **reconcile** drift continuously.

1 The duplicated listing

A seller creates a listing, is redirected to their shop page (built from search), and the listing isn't there. They create it again; now there are two. The first save worked perfectly. It just wasn't **searchable yet**. Users only notice the gap when they look for something they just did, so that is where the fix belongs.

2 Where the gap comes from

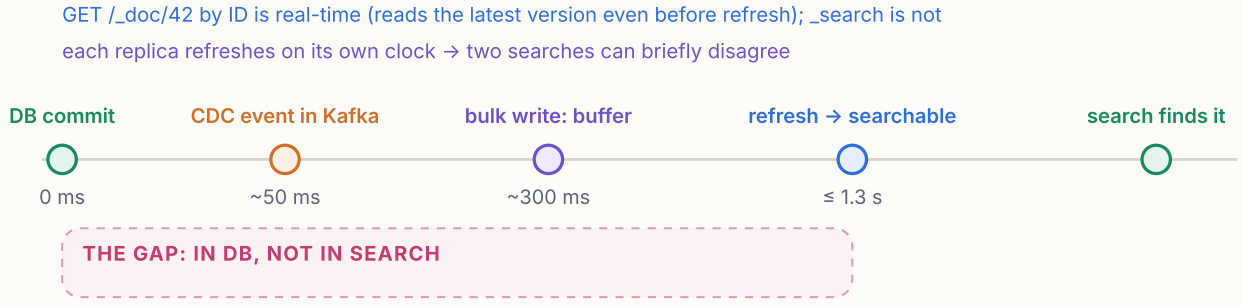


Figure 1. Typical timeline for a CDC pipeline (illustrative numbers). Most of the gap is pipeline lag plus the refresh interval.

Inside the engine, an indexed document sits in an in-memory buffer until a **refresh** (default every 1 s) writes a new searchable segment (lesson 15). With a CDC pipeline in front (lesson 19), add capture, queue and batching delay: usually a second or two, minutes if the indexer is lagging. Three subtleties catch people out:

- **Get vs search**: `GET index/_doc/42` is real-time by default, so 'it's there when I fetch it by ID' doesn't prove a search will find it.
- **Replicas refresh independently**, so consecutive searches routed to different copies can return different results. A `preference` value (e.g. the session ID) sends one user to the same copies for a consistent view.
- **Search-idle shards** skip scheduled refreshes when nobody searches them for 30 s; the first search afterwards waits for a refresh.

3 Forcing visibility, and what it costs

Option	Behaviour	Cost	Use when
`refresh=false` (default)	returns at once; visible after next refresh	none	almost always
`refresh=wait_for`	returns only after a refresh has made it visible	request waits up to the refresh interval	low-volume, user-facing writes
`refresh=true`	forces an immediate refresh of the affected shards	tiny segments, more merging, lower throughput	tests; rarely in production
shorter `refresh_interval`	whole index fresher	indexing throughput drops for every write	small, read-heavy indexes

`wait_for` doesn't create segments itself; it parks the request until the scheduled refresh. If too many requests wait (`index.max_refresh_listeners`, default 1,000) Elasticsearch forces a refresh. It also only helps when the client writes to the engine directly; with an async pipeline the app's request has already returned, so you need the product-side fixes below.

4 Read your own writes in the product

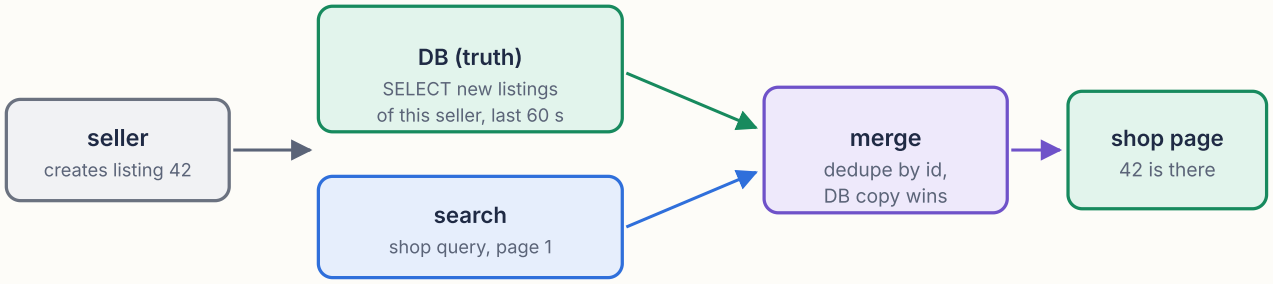


Figure 2. Merge the user's own recent writes from the database into search results until the index catches up.

- **Confirmation from the record**: after 'Create', show a page built from the saved row or the API response, not a search.
- **Merge recent writes**: the shop page asks the DB (or a short-lived cache keyed by user) for that seller's items created in the last minute and merges them into the search results, deduplicating by ID.
- **Optimistic UI**: insert the new item into the client-side list immediately.
- **Idempotency key** on the create endpoint, so a nervous double-submit can't create two listings anyway.

5 Deletes and authority

A deleted document is only marked deleted and disappears from results at the next refresh; behind an async pipeline it can linger for seconds or longer. So treat search results as **candidates**: on click, load the item from the database and show 'no longer available' if it's gone. Never use the index alone for **permissions** (a revoked share could still be visible), **stock** or **price at checkout**; re-check against the source of truth. For document-level security, filter by the user's tenant/ACL in the query (lesson 43) *and* verify on fetch. One more trap: if updates can arrive after a delete, a late update can recreate the document (see lesson 19); soft deletes (`deleted: true` plus a filter) avoid this.

6 Reconciling drift

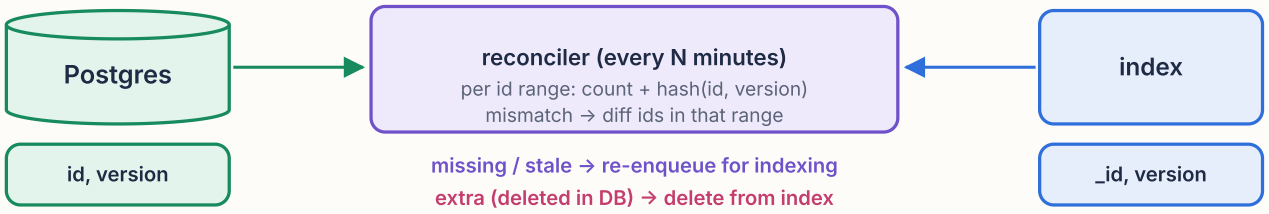


Figure 3. A reconciliation job turns silent drift into a bounded, self-healing delay.

Even a good pipeline drifts: an event dropped by a bug, a document rejected by a mapping error, a manual fix to the DB that bypassed CDC. Run a **reconciler**: split the ID space into ranges, compare per range a count and a hash of (id, version) from the DB and from the index; only for mismatched ranges, list IDs and diff them. Re-enqueue missing or stale IDs and delete extras. Track the **drift rate** as a metric: a rise points to a pipeline bug. Periodic full rebuilds behind an alias (lesson 19) are the heavy-handed version of the same idea.

WORKED EXAMPLE: THE MISSING NEW LISTING

The seller saves listing 42. The API commits it to Postgres and returns 201 with the saved record; the CDC pipeline indexes it about a second later. Instead of redirecting to a search-based page, the app shows a confirmation built from the record. The shop page runs the normal search *plus* `SELECT … WHERE seller_id = ? AND created_at > now() - interval '60 seconds'`, merges by ID and shows 42 at the top. A unique idempotency key on 'create listing' blocks duplicates. By the time the seller searches again, the index has caught up and the merge adds nothing. Cost: one indexed DB query on one page, instead of slowing every write with forced refreshes.

TRADE-OFFS

- **wait_for**: simple and correct for the writer, but adds up to a second of latency per write.
- **Read-your-writes merging**: no engine cost, but extra code and a DB query on specific pages.
- **Faster refresh**: fresher for everyone, but lower indexing throughput and more merging.
- **Frequent reconciliation** shortens drift but adds load on both the DB and the cluster.

COMMON MISTAKES

- Using `refresh=true` on every write to 'fix' consistency, then wondering why indexing collapsed.
- Redirecting to a search-backed page straight after a create.
- Trusting search results for access control or stock.
- Having no reconciliation, so a single lost event stays wrong forever.

7 Key terms

Near real time: searchable after the next refresh, not instantly

refresh=wait_for: write returns once it is searchable

Read-your-writes: a user always sees their own changes

Real-time GET: fetch by ID sees unrefreshed writes

preference: routes a user to the same shard copies

Reconciliation: periodic DB vs index diff and repair

8 Interview questions

Q1. How do you handle a user not seeing their newly created item in search? BASIC

Use refresh=wait_for where the write path can afford it, or better, read your own writes: show the item from the database or merge it into results until the index catches up.

Q2. How do you keep search and the database consistent? BASIC

Accept eventual consistency, make the pipeline idempotent and versioned, handle deletes carefully, and reconcile drift with periodic checks.

Q3. A user refreshes the results page and an item appears, disappears, then appears. Why? SCENARIO

Requests hit different replicas that refreshed at different moments; use a preference key (e.g. session ID) so one user sticks to the same copies.

Q4. A permission was revoked but the document still appears in another user's search. What's the design fix? SCENARIO

Search results are not authoritative: filter by ACL in the query and re-check permissions in the DB when the item is opened.

Q5. Why not just set refresh=true on all writes? TRICKY

Each forced refresh creates a tiny segment, which multiplies merge work and slashes indexing throughput; it only fixes visibility for direct writers anyway.

Q6. GET by ID returns the document but search doesn't. Contradiction? TRICKY

No: GET is real-time (it can read the latest version before a refresh), while search only sees refreshed segments.

Next: SD-P-008 · Autocomplete backendroadmap.com